GUIA DO PROFESSOR

ESP32-CAM: da câmera de segurança ao reconhecimento de gestos

Plano de 5 aulas — hardware, redes, nuvem, visão computacional e IA; gabarito dos desafios, solução de problemas, ética e avaliação

---

Placa: ESP32-CAM AI-Thinker (OV3660/OV2640) • **PC:** Python 3 + OpenCV 5 + MediaPipe 1.0

Público: Ensino Médio / Técnico (informática, eletrônica) • **Carga horária:** 5 aulas de 100 min

Espaço Cria — 2026

Sumário

1. Visão geral
2. Materiais
3. Preparação
4. Plano das aulas
5. Fundamentação para o professor
6. Erros comuns e como resolver
7. Gabarito e comentários dos desafios
8. Avaliação
9. Diferenciação
10. Segurança, privacidade e acordo da turma
11. Referências

Anexo A — Cabeçalho da câmera

Anexo B — Script final de referência

1. Visão geral

A sequência usa o ESP32-CAM para percorrer, de forma concreta, a cadeia completa de um sistema de visão: **sensor → compressão → rede local → armazenamento → nuvem → processamento no PC → decisão → atuador**. É a aula com mais "camadas" do programa e a que mais exige preparo: gravação sem USB, alimentação, contas de serviço (Google, Telegram) e ambiente Python com modelos. Em troca, é a que produz o maior impacto: os alunos veem a própria mão controlando um LED por meio de uma rede neural, e entendem cada elo do caminho.

Objetivos de aprendizagem

- Descrever o hardware do ESP32-CAM (câmera DVP, PSRAM, SD, flash, pinos livres, strapping) e gravar firmware com adaptador serial.
- Explicar a formação da imagem digital (fotodiodo, Bayer, exposição, JPEG) e as diferenças OV2640 × OV3660.
- Implementar captura, MJPEG e controle por HTTP; gravar em SD com nomes por tempo; construir uma galeria web.
- Distinguir rede local × internet (NAT) e escolher uma estratégia de publicação (túnel × envio para a nuvem); usar HTTPS, base64 e multipart.
- Representar imagens como matrizes; aplicar operações clássicas (cinza, bordas, HSV, contornos, Haar).
- Usar um modelo de ML pré-treinado (MediaPipe) e integrá-lo a um atuador com estabilização e watchdog.
- Discutir privacidade, consentimento e segurança de câmeras conectadas.

Competências (BNCC / Itinerários)

- **Física:** efeito fotoelétrico, radiação infravermelha, óptica (lente, campo de visão), amostragem.
- **Matemática:** matrizes, médias ponderadas (cinza), gradientes (bordas), logaritmos/compressão, estatística de votação.
- **Computação:** protocolos (HTTP, multipart), codificação (base64, JPEG), sistemas distribuídos (cliente-servidor, nuvem), IA aplicada (inferência), threads.
- **Cidadania digital:** privacidade, consentimento, LGPD, riscos de dispositivos expostos.

Pré-requisitos

Sequências anteriores: página web no ESP32 (`WebServer`/rotas), `millis()`, LED RGB, buzzer, PIR (opcional), Python básico com serial (aula de distância). Sem elas, reserve 30 min extras na Aula 1 e 3.

2. Materiais

Por grupo

Item	Qtd	Custo aprox. (R$)	Observação
ESP32-CAM AI-Thinker com OV3660 (ou OV2640)	1 (+1	45–70	Conferir se tem PSRAM (quase

Item	Qtd	Custo aprox. (R$)	Observação
	reserva/turma)		todos têm)
Base ESP32-CAM-MB (micro-USB) **ou** adaptador FTDI/CH340 5 V + jumper	1	15–25	A base evita o ritual do GPIO 0
Fonte/porta USB ≥ 1 A; cabo curto	1	—	Brownout é o erro nº 1
microSD 4–32 GB FAT32	1	20	Formatar antes
PIR HC-SR501	1	8–12	Aula 2–3
LED RGB + 3 × 220 Ω; LED verde + 220 Ω; buzzer; botão	1 kit	10	Desafios
Protoboard + jumpers macho-fêmea	1	25	Fêmea para os pinos do ESP32-CAM
PC com Python 3 (OpenCV, MediaPipe, requests), webcam opcional	1	—	Aulas 4–5

Para a turma

- Conta Google **do laboratório** (Drive + Apps Script) e um bot do Telegram do laboratório; rede Wi-Fi com internet (ou hotspot) para a Aula 3.
- Projetor; objetos coloridos (bolas/cartões vermelho, verde, azul) para a Aula 4; boa iluminação frontal para a Aula 5.
- Termo de consentimento simples para uso de imagem nas atividades (modelo no §10).

3. Preparação

1. IDE com core **esp32 ≥ 3.0**; placa **AI Thinker ESP32-CAM** (partição padrão de 3 MB já serve; os 4 sketches usam 32–35 %). Nenhuma biblioteca extra.
2. Todos os sketches foram **compilados** (01–04). Copie `aula-esp32cam/codigo/` (cada pasta já contém `camera_aithinker.h`) e `aula-esp32cam/python/` (com `modelos/`: `gesture_recognizer.task`, `hand_landmarker.task`, `haarcascade_frontalface_default.xml`).
3. Python: `pip install opencv-python mediapipe requests numpy` (testado com OpenCV 5.0 e MediaPipe 1.0.1 — a API é a `tasks`, não a antiga `solutions`). Teste sem hardware: `python reconhecer_imagem.py --demo` e `python gestos_leds.py --webcam --sem-esp`.
4. **Aula 3:** publique o `Code.gs` (Implantar → App da Web → executar como você → acesso: qualquer pessoa) e teste a URL `/exec` no navegador; crie o bot no BotFather e obtenha o chat_id; grave o sketch 03 com as credenciais e confirme uma foto no Drive e no Telegram **antes** da aula. Redes escolares podem bloquear `script.google.com`/`api.telegram.org`: tenha um hotspot.
5. Grave o sketch 01 em todos os ESP32-CAM na véspera e teste o stream: separa placas com câmera/PSRAM defeituosa.
6. Esquemas Fritzing em `aula-esp32cam/esquemas/saida/*.fzz` (peças ESP32-CAM e PIR instaladas em `Documents/Fritzing/parts/user`).

4. Plano das aulas

Aula 1 — ESP32-CAM e OV3660

Tempo	Atividade	Professor	Alunos	Evidência
10'	Abertura	Demonstra o projeto final (gesto → LED). “Como a câmera vira números?”	Hipóteses	—
20'	Sensor CMOS e JPEG	Fotodiodo, Bayer, exposição, JPEG; OV2640 × OV3660; PSRAM. Mostra um JPEG em hexadecimal	Anotam	—
15'	Anatomia e gravação	Pinos, strapping, o ritual do GPIO 0 (ou a base MB); 5 V	Gravam o sketch 01	Serial: “Camera OK: sensor OV3660”
20'	Foto e stream	Explica MJPEG e os dois servidores; `/capture` no navegador	Veem o vídeo no celular	Stream no celular
15'	Experimente	fps × resolução; flash; rolling shutter	Medem e anotam	Tabela fps
15'	Desafios 1.1/1.5	Botão de foto; bipe de “ao vivo”	Programam	Sketch
5'	Ticket	“Por que devolver o frame buffer?”	Respondem	Ticket

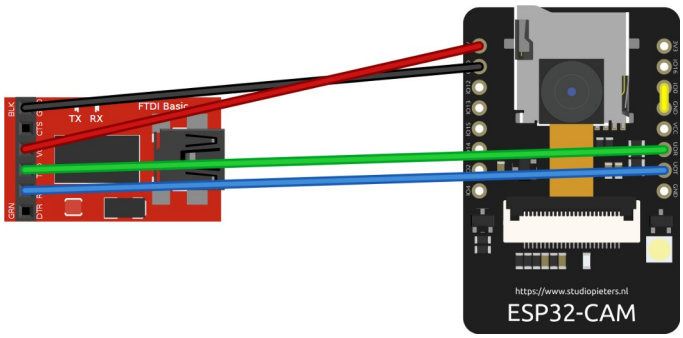


Figura 1 — Aula 1 — gravação com adaptador serial.

Aula 2 — Câmera de segurança

Tempo	Atividade	Professor	Alunos	Evidência
10'	Problema	“O que uma câmera de segurança precisa fazer sozinha?” (detectar, registrar, mostrar)	Listam requisitos	Lista
15'	PIR	Piroelétrico + Fresnel; demonstra: parado × movendo; vidro bloqueia	Testam alcance	Medidas
20'	Montagem + sketch 02	SD 1-bit e pinos livres; NTP × modo AP; borda + intervalo	Montam; gravam	Foto no SD

Tempo	Atividade	Professor	Alunos	Evidência
20'	Galeria e timelapse	`/lista` JSON + `/foto` em chunks; timelapse 10 s	Usam a galeria	Galeria com fotos
25'	Desafios 2.1/2.2	Alarme com buzzer/RGB e botão armar; rajada	Programam	Sketch
10'	Ética	Onde uma câmera pode apontar? Consentimento; retenção das imagens	Debate	Acordo da turma

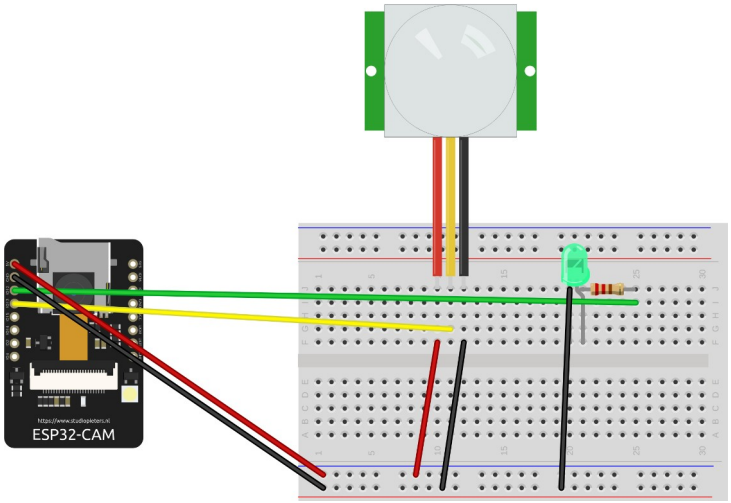


Figura 2 — Aula 2 — PIR e LED de status.

Aula 3 — Internet: Drive e Telegram

Tempo	Atividade	Professor	Alunos	Evidência
15'	Rede local × internet	NAT no quadro; 3 caminhos (port-forward, túnel, envio); riscos	Desenham	Desenho
15'	Apps Script	Mostra o `Code.gs` publicado; testa a URL; explica base64	Acompanham	—
15'	Telegram	BotFather, chat_id, `sendPhoto` e multipart no Serial	Configuram (credenciais do lab)	—
20'	Sketch 03	Grava; movimento → Drive + Telegram; `ultima.jpg` no celular	Testam	Foto no Drive e no celular
25'	Desafios 3.1/3.2	Comando de volta por `getUpdates`; regra de 3 detecções	Programam	Sketch
10'	Fechamento	"Por que o stream não sai da sala?"	Respondem	Ticket

Aula 4 — Python vê imagens

Tempo	Atividade	Professor	Alunos	Evidência
10'	Imagem = matriz	`img.shape`, `img[0,0]`, BGR; pixel art no quadro	Rodam `--demo`	—
20'	Cinza, bordas, HSV	Fórmula do cinza; gradiente; roda HSV; por que o vermelho tem duas faixas	Alteram limiares	Janelas
15'	`ver_stream.py`	VideoCapture com MJPEG; salvar foto/vídeo	Veem a câmera no PC	Vídeo gravado
15'	Contornos e Haar	Contorno/caixa; cascata: padrões claro-escuro; limites	Testam rostos	—
30'	Desafios 4.1/4.4	Semáforo de cor → LED; detector de movimento por `absdiff` × PIR	Programam	Script
10'	Fechamento	Visão clássica × aprendizado de máquina	Discutem	Ticket

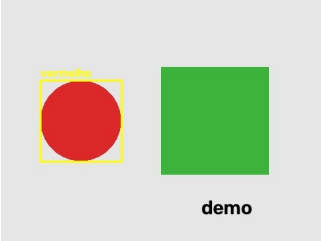


Figura 3 — Aula 4 — resultado do demo.

Aula 5 — Gestos → LEDs

Tempo	Atividade	Professor	Alunos	Evidência
15'	MediaPipe	Detector de palma → 21 landmarks → classificador; inferência × treinamento; confiança	Rodam `--webcam --sem-esp`	Gestos na tela
15'	Montagem + sketch 04	RGB nos GPIO 12/13/14; rotas `/led`, `/estado`; watchdog	Montam; gravam	Página com bolinha de cor
20'	Integração	Câmera → PC → ESP32; votação; thread; latência medida	Fazem os 7 gestos	LED muda com o gesto
30'	Desafios 5.2/5.3	Senha por gestos (máquina de estados); contagem de dedos	Programam	Script + sketch
15'	Apresentação	Cada grupo demonstra e explica um elo da cadeia	Apresentam	Rubrica
5'	Fechamento	"O que muda se o modelo rodasse no ESP32?"	Respondem	Ticket

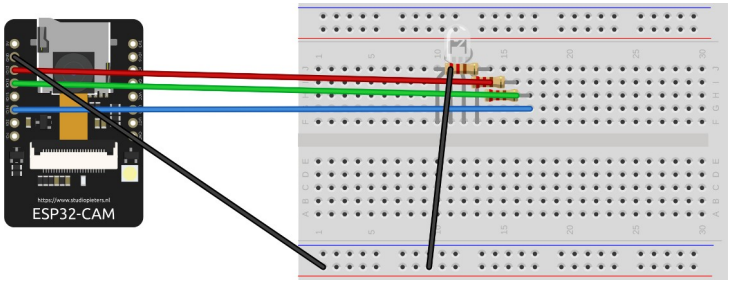


Figura 4 — Aula 5 — LED RGB no ESP32-CAM.

5. Fundamentação para o professor

ESP32-CAM: hardware e armadilhas

A câmera usa interface **DVP** paralela (8 bits de dados + VSYNC/HREF/PCLK + XCLK de 20 MHz gerado pelo ESP32) e **SCCB** (I^2C) para configuração. Isso consome 16 GPIOs; sobram 12, 13, 14, 15 (SD 4-bit), 2 (SD 1-bit D0), 4 (flash/SD D1), 16 (PSRAM CS em algumas placas — evite), 33 (LED vermelho) e a UART 0. **Strapping**: GPIO 0 baixo no boot = modo de gravação; GPIO 12 alto no boot = tensão da flash em 1,8 V (a placa não sobe) — LED para o GND é seguro, pull-up não. **Alimentação**: picos de 300–400 mA com Wi-Fi + câmera; o regulador AMS1117 esquenta; FTDI 3,3 V ou porta USB fraca causam "Brownout detector was triggered". **PSRAM**: sem ela, `FRAMESIZE` acima de VGA falha na inicialização (erro 0x105/0x20004).

OV3660 × OV2640

Ambos são sensores OmniVision com JPEG embutido; o driver `esp32-camera` detecta pelo PID. OV3660: 3 MP (QXGA), melhor em baixa luz, imagem invertida por padrão (`set_vflip(1)`), tende a saturar cores (daí `set_saturation(-2)`); XCLK de 20 MHz funciona, 10 MHz é mais estável em placas com cabos longos. UXGA/QXGA a ~2–5 fps; VGA ~20 fps; QVGA ~30 fps (limitados pelo Wi-Fi e pela compressão). Qualidade JPEG 10–12 ≈ 40–80 kB em SVGA.

MJPEG, HTTP e concorrência

O `esp_http_server` roda em tarefa própria; um handler de stream ocupa a conexão indefinidamente, por isso o segundo servidor na porta 81 (padrão do exemplo da Espressif). Cada cliente do stream consome um `esp_camera_fb_get` por quadro; dois clientes dividem os fps. `CAMERA_GRAB_LATEST` + `fb_count = 2` evita quadros atrasados. Para muitos clientes ou vídeo real (H.264), o ESP32 não serve — use ESP32-S3 ou um Raspberry Pi.

SD, NTP e sistema de arquivos

`SD_MMC` é mais rápido que `SD` (SPI) e usa o periférico dedicado; em 1-bit chega a ~1 MB/s, suficiente para JPEGs. Nomes por tempo exigem relógio: NTP só com internet; alternativa, um RTC DS3231 por I^2C nos GPIO 14/15. FAT32 limita a 32 GB e não gosta de remoção com escrita em andamento (LED de status ajuda). Não há vídeo AVI nativo; projetos comunitários (ESP32-CAM-Video-Recorder, de jameszah/s60sc) gravam AVI-MJPEG e podem ser apresentados como extensão.

Internet: NAT, túneis e nuvem

Port-forwarding expõe um dispositivo sem autenticação — não faça em ambiente escolar. Túneis (Tailscale/ngrok/Cloudflare Tunnel) resolvem o acesso remoto com autenticação e são o caminho profissional para "ao vivo". O envio para a nuvem (Aula 3) é o mais didático: mostra HTTPS, base64 (RFC 4648: 3 bytes → 4 caracteres, +33 %), multipart (RFC 7578) e o papel de um intermediário (Apps Script) que tem as credenciais do Drive — o ESP32 nunca vê a senha do Google. `setInsecure()` desliga a verificação de certificado (MITM possível); com `setCACert` e o certificado raiz do Google fica correto — bom tema para os avançados.

Visão clássica

Uma imagem 8-bit BGR é um `ndarray (h, w, 3)`; operações são álgebra de matrizes (por isso numpy). Cinza: luminância ITU-R 601. Canny: suavização gaussiana → gradiente (Sobel) → supressão de não-máximos → histerese com dois limiares. HSV separa crominância de luminância, tornando a segmentação por cor robusta à iluminação. Haar cascades (Viola-Jones, 2001): características retangulares + AdaBoost + cascata; rápido, mas sensível a pose e iluminação — motiva o salto para redes neurais. No OpenCV 5 os XML não vêm no pacote (por isso o arquivo em `modelos/`).

MediaPipe e inferência

O `GestureRecognizer` encadeia três modelos TFLite (palma, landmarks, gesto) e roda em CPU a 20–60 fps em VGA. Os 21 landmarks são normalizados (0–1) e têm profundidade relativa. As 7 categorias são fixas no modelo padrão; gestos novos exigem *Model Maker* (treinamento com fotos rotuladas). A votação de 4/5 quadros é um filtro de moda — análogo à mediana dos sensores. O watchdog de 5 s no ESP32 é uma prática de segurança funcional: atuadores não devem ficar num estado se o controle sumir. Latência típica do sistema: 150–400 ms (captura ~80 ms + rede ~30 ms + inferência ~30 ms + HTTP de volta ~50 ms).

Ética e privacidade

Imagens de pessoas são dados pessoais (LGPD). Combine com a turma: câmeras apontam para objetos/mesas, ou para quem consentiu; fotos no Drive/Telegram do laboratório são apagadas ao fim; nada de publicar. A Aula 2 traz um debate de 10 min e o §10 tem um modelo de acordo.

6. Erros comuns e como resolver

Sintoma	Causa provável	Solução
“Failed to connect to ESP32” ao gravar	IO0 não aterrado; RST não apertado; TX/RX trocados; adaptador em 3,3 V	Ritual IO0→RST→Upload; conferir cruzamento; 5 V
“Brownout detector was triggered” / reinicia ao ligar Wi-Fi	Alimentação fraca	Fonte 5 V ≥ 1 A; cabo curto; capacitor 470 µF no 5 V
“Camera init failed 0x105/0x20004”	Cabo flat solto/invertido; PSRAM ausente com resolução alta; placa clone com outros pinos	Reencaixar flat; VGA; conferir modelo
Imagem verde/rosada ou de cabeça para baixo	OV3660 sem os ajustes	`set_vflip(1)`, `set_saturation(-2)` (já no header)
Stream trava após alguns segundos	Frame buffer não devolvido; 2 clientes; Wi-Fi fraco	`esp_camera_fb_return`; um cliente por vez; aproximar
“Camera capture failed” intermitente	XCLK 20 MHz instável / cabo longo	`xclk_freq_hz = 10000000`
SD não monta	Não é FAT32; > 32 GB; contatos sujos; modo 4-bit com GPIO 4 em uso	Formatar FAT32; cartão menor; `begin("/sdcard", true)`
PIR dispara sozinho	Estabilização (60 s); sensibilidade alta; calor do ESP32/flash perto	Esperar; ajustar pot; afastar

Sintoma	Causa provável	Solução
Drive: HTTP 302 / “faltou o parametro”	Sem `setFollowRedirects`; implantação antiga; corpo > limite	Seguir redirect; nova versão da implantação; VGA
Drive: HTTP 401/403	App da Web não é “qualquer pessoa” ou não autorizado	Reimplantar com acesso “qualquer pessoa”; autorizar
Telegram: 400/401	Token errado; chat_id errado; bot nunca recebeu “oi”	Conferir; mandar mensagem ao bot
Python: “nao abriu o stream”	PC não está na rede da câmera; URL sem `:81`	Conectar; `http://IP:81/stream`
OpenCV: cascade `!empty()`	XML ausente (OpenCV 5)	`modelos/haarcascade_frontalface_default.xml`
MediaPipe: `AttributeError solutions`	API antiga em tutoriais da web	Usar `mediapipe.tasks.python.vision` (scripts do curso)
Gesto oscila / LED pisca entre cores	Sem votação; luz de costas; mão pequena no quadro	Manter votação 4/5; luz frontal; aproximar; VGA
LED apaga sozinho a cada 5 s	Watchdog: Python parou de enviar	É esperado; conferir o script/rede

7. Gabarito e comentários dos desafios

Aula 1

- **1.1** `pinMode(13, INPUT_PULLUP)`; na borda: flash HIGH, `esp_camera_fb_get`, gravar/enviar, `fb_return`, flash LOW; `digitalWrite(33, LOW)` pisca (ativo em LOW).
- **1.2** Estimativa por tamanho do JPEG é uma heurística válida (imagem escura comprime mais); alternativa: `s->set_gain_ctrl` e ler `s->status`. Discutir limitações.
- **1.3** `s->set_hmirror(s, 1)`; zoom digital via `set_res_raw` é avançado — aceitar solução com resolução menor.
- **1.4** Incrementar/decrementar um contador global na entrada/saída do `streamHandler`; expor em `/estado`.
- **1.5** `tone(14, 2000, 100)` na entrada do handler (o core 3.x tem `tone` para ESP32).

Aula 2

- **2.1** Estados ARMADO/DESARMADO por botão (borda + debounce); em ARMADO, PIR → 3 bipes + RGB vermelho por 10 s (`millis()`); LED de estado verde/vermelho.
- **2.2** `for (i<3) { tirarFoto(); delay(1000); }` — discutir por que o `delay` aqui é aceitável (evento raro) e como faria sem.
- **2.3** Ao gravar, se `numFotos > 200`, listar e `SD_MMC.remove` o nome mais antigo (ordem alfabética = cronológica).
- **2.4** `getLocalTime(&t); bool vigilancia = t.tm_hour >= 18 || t.tm_hour < 7;`.
- **2.5** Vetor `int eventos[24]`, incrementar `eventos[t.tm_hour]`; rota `/histograma` JSON; barras com `<div>` de altura proporcional.

Aula 3

- **3.1** `GET /bot<TOKEN>/getUpdates?offset=<ultimo+1>` a cada 2 s; parse simples com `indexOf("\"text\":\"")`; responder com `sendMessage`/`sendPhoto`. Introduz polling de API.
- **3.2** Vetor circular dos últimos 3 instantes de PIR; só envia se `millis() - t[0] < 60000`.
- **3.3** Contador diário + `tm_hour == 18 && !enviadoHoje`; reutilizar `enviarEmail` da aula de distância.
- **3.4** No `Code.gs`: `const hoje = Utilities.formatDate(new Date(), tz, "yyyy-MM-dd"); let sub = pasta.getFoldersByName(hoje); sub = sub.hasNext() ? sub.next() : pasta.createFolder(hoje);`.
- **3.5** Tailscale no PC + `tailscale serve`/subnet router; discutir autenticação e por que não abrir a porta 81 no roteador.

Aula 4

- **4.1** Três máscaras HSV; escolher a de maior área; `requests.get(f"http://{ESP}/led", params={"cor": cor})` — o sketch 04 já responde.
- **4.2** `len([c for c in contornos if cv2.contourArea(c) > 500])`; `winsound.Beep`.
- **4.3** `sorted(glob("*.jpg"))` → `VideoWriter(..., 10 fps, (w, h))`.

- **4.4** `diff = cv2.absdiff(anterior, atual)` em cinza → `threshold(25)` → área de pixels brancos > 2 %; PIR detecta calor em movimento (funciona no escuro), a diferença de quadros detecta qualquer mudança (inclusive luz).
- **4.5** `len(rostos)` → 0/1/≥2 → cor; enviar só quando muda.

Aula 5

- **5.1** `num_hands=2`; `res.handedness[i][0].category_name` diz Left/Right; brilho por gesto da direita (`/led?r=..` com fator).
- **5.2** Máquina de estados com índice na sequência e temporizador de 5 s; qualquer gesto fora da ordem zera; LED verde + `tone` no ESP32 (rota `/ok`).
- **5.3** Com `HandLandmarker`: dedo levantado se `landmark[ponta].y < landmark[articulacao].y` (indicador 8/6, médio 12/10, anelar 16/14, mínimo 20/18; polegar pelo x). Contar → `/led?piscar=n`.
- **5.4** HTTP GET ao ESP32 do braço (`/pose?n=1`) — integração entre projetos; discutir endereçamento (dois APs não conversam: usar modo estação na mesma rede).
- **5.5** A Espressif fornece detecção de rostos (esp-dl/esp-face) no próprio ESP32 em QVGA a ~5 fps; classificação de gestos não cabe na RAM/tempo. O ESP32-S3 com PSRAM octal roda modelos TFLite Micro pequenos.

8. Avaliação

Rubrica do projeto final

Critério	Iniciante (1)	Em desenvolvimento (2)	Proficiente (3)
Hardware e gravação	Precisa de ajuda para gravar	Grava e explica pinos/strapping/alimentação	Diagnostica brownout/flat/PSRAM sozinho
Imagem e rede	Stream funciona	Explica MJPEG, JPEG, resolução × fps	Mede e otimiza; explica portas 80/81 e concorrência
Armazenamento e nuvem	Grava no SD	Galeria + Drive/Telegram funcionando	Explica NAT, base64, multipart, cooldown; credenciais seguras
Visão computacional	Roda os scripts	Explica matriz, cinza, HSV, contornos	Cria um detector próprio (cor/movimento) e integra ao LED
IA e integração	Gesto acende LED	Explica landmarks, confiança, votação, watchdog	Máquina de estados por gestos ou contagem de dedos; mede latência
Ética e comunicação	—	Segue o acordo de privacidade	Justifica escolhas de privacidade; apresentação clara da cadeia completa

Formativa

- Tickets de saída (nos planos).

- Tabela fps × resolução (Aula 1).
- Galeria e nomes de arquivo (Aula 2).
- Prova de envio (print do Drive/Telegram) (Aula 3).
- Scripts comentados (Aulas 4–5).
- Acordo de privacidade assinado pela turma.

9. Diferenciação

- **Avançados:** 1.3 (zoom digital), 3.1 (Telegram bidirecional), 3.5 (túnel), 4.4 (movimento), 5.2/5.3 (estados, landmarks), `setCACert`.
- **Dificuldade:** base ESP32-CAM-MB (sem ritual do GPIO 0); scripts com `--webcam --sem-esp`; um ESP32-CAM de demonstração por sensor de conteúdo; pares com um aluno de informática.
- **Sem internet:** Aula 3 vira demonstração do professor com hotspot + discussão; alunos fazem o desafio 2.5.
- **Sem PC potente:** MediaPipe roda em CPU comum (i3/Ryzen 3); reduzir para QVGA se necessário.

10. Segurança, privacidade e acordo da turma

- Alimentação 5 V dimensionada; nada de pull-up no GPIO 12; flash não apontado aos olhos.
- Câmeras apontam para bancadas/objetos; pessoas só com consentimento verbal registrado.
- Contas de Google/Telegram do laboratório; credenciais distribuídas na aula e revogadas depois (regenerar token no BotFather; nova implantação do Apps Script).
- Imagens apagadas do Drive/Telegram/SD ao final do curso.
- **Modelo de acordo:** “Nós, alunos da turma ___, combinamos que as câmeras das atividades só registram o que cada grupo autorizou, que nenhuma imagem será publicada fora do laboratório e que todas serão apagadas em __/__/__.”

11. Referências

- Espressif — esp32-camera (driver) e exemplo CameraWebServer: https://github.com/espressif/esp32-camera
- OmniVision OV3660 / OV2640 — datasheets.
- Random Nerd Tutorials — ESP32-CAM (gravação, SD, PIR, Telegram, Drive).
- Google Apps Script — Web Apps: https://developers.google.com/apps-script/guides/web
- Telegram Bot API — sendPhoto/getUpdates: https://core.telegram.org/bots/api
- OpenCV docs: https://docs.opencv.org/ • MediaPipe Gesture Recognizer: https://ai.google.dev/edge/mediapipe/solutions/vision/gesture_recognizer
- LGPD — Lei 13.709/2018 (dados pessoais e imagem).
- Códigos deste curso: `aula-esp32cam/codigo/` e `aula-esp32cam/python/`.

Anexo A — Cabeçalho da câmera

camera_aithinker.h

```
/*
  camera_aithinker.h - inicializacao da camera do ESP32-CAM (AI-Thinker) com sensor OV2640 ou OV3660.
  Copie este arquivo para a pasta de cada sketch. Uso:
      #include "camera_aithinker.h"
      void setup() { ... if (!cameraIniciar()) { ... } }

  Pinos da camera no AI-Thinker (fixos, definidos pela placa):
     XCLK 0, SIOD 26, SIOC 27, Y9..Y2 = 35 34 39 36 21 19 18 5, VSYNC 25, HREF 23, PCLK 22, PWDN 32
  Pinos LIVRES para o seu projeto: 12, 13, 14, 15 (se o microSD nao for usado), 2 (SD 1-bit), 4 (flash),
16 e 33 (LED vermelho, ativo em LOW)
*/
#pragma once
#include "esp_camera.h"

#define PWDN_GPIO_NUM   32
#define RESET_GPIO_NUM  -1
#define XCLK_GPIO_NUM    0
#define SIOD_GPIO_NUM   26
#define SIOC_GPIO_NUM   27
#define Y9_GPIO_NUM     35
#define Y8_GPIO_NUM     34
#define Y7_GPIO_NUM     39
#define Y6_GPIO_NUM     36
#define Y5_GPIO_NUM     21
#define Y4_GPIO_NUM     19
#define Y3_GPIO_NUM     18
#define Y2_GPIO_NUM      5
#define VSYNC_GPIO_NUM  25
#define HREF_GPIO_NUM   23
#define PCLK_GPIO_NUM   22
#define FLASH_GPIO_NUM   4     // LED branco de alta potencia
#define LED_VERMELHO    33     // LED pequeno no verso (LOW = aceso)

const char* cameraNome = "?";

bool cameraIniciar(framesize_t tamanho = FRAMESIZE_SVGA, int qualidade = 12) {
  camera_config_t c = {};
  c.ledc_channel = LEDC_CHANNEL_0;  c.ledc_timer = LEDC_TIMER_0;
  c.pin_d0 = Y2_GPIO_NUM; c.pin_d1 = Y3_GPIO_NUM; c.pin_d2 = Y4_GPIO_NUM; c.pin_d3 = Y5_GPIO_NUM;
  c.pin_d4 = Y6_GPIO_NUM; c.pin_d5 = Y7_GPIO_NUM; c.pin_d6 = Y8_GPIO_NUM; c.pin_d7 = Y9_GPIO_NUM;
  c.pin_xclk = XCLK_GPIO_NUM; c.pin_pclk = PCLK_GPIO_NUM; c.pin_vsync = VSYNC_GPIO_NUM; c.pin_href =
HREF_GPIO_NUM;
  c.pin_sccb_sda = SIOD_GPIO_NUM; c.pin_sccb_scl = SIOC_GPIO_NUM; c.pin_pwdn = PWDN_GPIO_NUM; c.pin_reset
= RESET_GPIO_NUM;
  c.xclk_freq_hz = 20000000;          // 20 MHz para o sensor
  c.pixel_format = PIXFORMAT_JPEG;    // o sensor ja entrega JPEG comprimido
  c.frame_size = tamanho;             // SVGA 800x600 e um bom equilibrio; OV3660 vai ate QXGA 2048x1536
  c.jpeg_quality = qualidade;         // 0 (melhor) .. 63 (pior); 10-12 e bom
  c.fb_count = psramFound() ? 2 : 1;  // 2 buffers na PSRAM = video mais fluido
  c.fb_location = psramFound() ? CAMERA_FB_IN_PSRAM : CAMERA_FB_IN_DRAM;
  c.grab_mode = CAMERA_GRAB_LATEST;
  if (!psramFound()) { c.frame_size = FRAMESIZE_VGA; }

  esp_err_t err = esp_camera_init(&c);
  if (err != ESP_OK) { Serial.printf("Camera falhou: 0x%x (cabo flat? PSRAM? alimentacao 5 V?)\n", err);
return false; }

  sensor_t* s = esp_camera_sensor_get();
  if (s->id.PID == OV3660_PID) {      // ajustes recomendados para o OV3660
    cameraNome = "OV3660";
    s->set_vflip(s, 1);               // a imagem vem de cabeca para baixo neste sensor
    s->set_brightness(s, 1);
    s->set_saturation(s, -2);
  } else if (s->id.PID == OV2640_PID) cameraNome = "OV2640";
  Serial.printf("Camera OK: sensor %s, PSRAM %s\n", cameraNome, psramFound() ? "sim" : "NAO");
  return true;
}
```

Anexo B — Script final de referência

gestos_leds.py

```
"""Projeto: gestos com a mao (MediaPipe) acendem cores no LED RGB do ESP32-CAM.

Uso:
    python gestos_leds.py                                  # camera: ESP32-CAM em 192.168.4.1
    python gestos_leds.py --esp 192.168.1.50               # outro IP
    python gestos_leds.py --webcam                         # testa com a webcam do PC (sem ESP32)
    python gestos_leds.py --webcam --sem-esp               # so reconhece, nao manda nada
Tecla q sai.

Gestos que o modelo do Google reconhece -> cor enviada ao ESP32 (rota /led?cor=...):
    Open_Palm (mao aberta) -> verde      Closed_Fist (punho) -> vermelho   Pointing_Up (dedo p/ cima) ->
azul
    Victory (V) -> amarelo               Thumb_Up (joinha) -> roxo         ILoveYou -> ciano
    Thumb_Down (joinha p/ baixo) -> apaga
Bibliotecas: opencv-python, mediapipe, requests.  Modelo: modelos/gesture_recognizer.task
"""
import sys, os, time, threading, cv2, numpy as np, requests
import mediapipe as mp
from mediapipe.tasks.python import vision, BaseOptions

ESP = "192.168.4.1"
if "--esp" in sys.argv: ESP = sys.argv[sys.argv.index("--esp") + 1]
USAR_WEBCAM = "--webcam" in sys.argv
ENVIAR = "--sem-esp" not in sys.argv
MODELO = os.path.join(os.path.dirname(os.path.abspath(__file__)), "modelos", "gesture_recognizer.task")
CORES = {"Open_Palm": "verde", "Closed_Fist": "vermelho", "Pointing_Up": "azul", "Victory": "amarelo",
         "Thumb_Up": "roxo", "ILoveYou": "ciano", "Thumb_Down": "off"}
CORES_BGR = {"verde": (0, 255, 0), "vermelho": (0, 0, 255), "azul": (255, 0, 0), "amarelo": (0, 200, 255),
"roxo": (255, 0, 160), "ciano": (255, 255, 0), "off": (80, 80, 80)}

# ---------- fonte de video ----------
class FonteESP:
    """le fotos /capture em sequencia (mais robusto que o stream quando o Wi-Fi oscila)"""
    def __init__(self, ip): self.url = f"http://{ip}/capture"
    def ler(self):
        try:
            dados = requests.get(self.url, timeout=3).content
            return cv2.imdecode(np.frombuffer(dados, np.uint8), cv2.IMREAD_COLOR)
        except requests.RequestException: return None
class FonteWebcam:
    def __init__(self): self.cap = cv2.VideoCapture(0)
    def ler(self):
        ok, q = self.cap.read(); return q if ok else None

# ---------- envio ao ESP32 em segundo plano (nao trava o video) ----------
ultimaCorEnviada = None
def enviar(cor):
    global ultimaCorEnviada
    if not ENVIAR or cor == ultimaCorEnviada: return
    ultimaCorEnviada = cor
    def _go():
        try: requests.get(f"http://{ESP}/led", params={"cor": cor}, timeout=2)
        except requests.RequestException: print("(ESP32 nao respondeu)")
    threading.Thread(target=_go, daemon=True).start()

# ---------- reconhecedor ----------
opcoes = vision.GestureRecognizerOptions(base_options=BaseOptions(model_asset_path=MODELO), num_hands=1,
                                         min_hand_detection_confidence=0.5, min_tracking_confidence=0.5)
reconhecedor = vision.GestureRecognizer.create_from_options(opcoes)

def main():
    fonte = FonteWebcam() if USAR_WEBCAM else FonteESP(ESP)
    print("fonte:", "webcam" if USAR_WEBCAM else f"ESP32-CAM {ESP}", "| envio:", "sim" if ENVIAR else
"nao")
    historico = []                                         # os ultimos N gestos: so aceita se for estavel
    while True:
        quadro = fonte.ler()
        if quadro is None: time.sleep(0.3); continue
```

```
        rgb = cv2.cvtColor(quadro, cv2.COLOR_BGR2RGB)     # MediaPipe usa RGB; OpenCV usa BGR
        res = reconhecedor.recognize(mp.Image(image_format=mp.ImageFormat.SRGB, data=rgb))
        gesto, conf = "None", 0.0
        if res.gestures:
            g = res.gestures[0][0]; gesto, conf = g.category_name, g.score
            for lm in res.hand_landmarks[0]:              # desenha os 21 pontos da mao
                cv2.circle(quadro, (int(lm.x * quadro.shape[1]), int(lm.y * quadro.shape[0])), 4, (0, 255,
255), -1)
        historico.append(gesto); historico = historico[-5:]
        estavel = historico.count(gesto) >= 4             # 4 de 5 quadros iguais = gesto firme
        cor = CORES.get(gesto, None)
        if estavel and cor: enviar(cor)
        cv2.rectangle(quadro, (0, 0), (quadro.shape[1], 40), CORES_BGR.get(cor, (40, 40, 40)), -1)
        cv2.putText(quadro, f"{gesto} ({conf:.2f}) -> {cor or '-'}", (10, 28), cv2.FONT_HERSHEY_SIMPLEX,
0.8, (255, 255, 255), 2)
        cv2.imshow("gestos -> LEDs (q sai)", quadro)
        if cv2.waitKey(1) & 0xFF == ord("q"): break
    cv2.destroyAllWindows()

if __name__ == "__main__":
    main()
```